Université Abdelmalek Essaâdi
École Nationale des Sciences Appliquées – Al Hoceima
TP – J2EE, Java EE (JEE) ou Jakarta EE
TDIA2-S4 / 2025-26
Pr. Mohamed CHERRADI

TP N° 4:

Réalisés par: Asmae Daouad.

Introduction et objectifs:

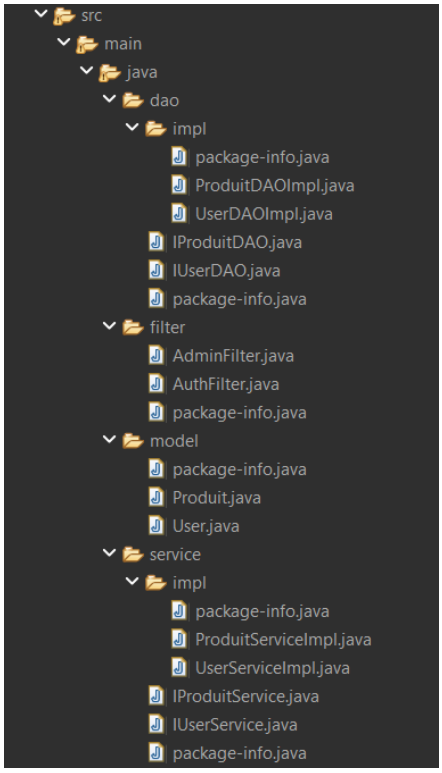
Ce travail pratique consiste à développer une application web Java EE respectant l'architecture MVC1 (Model-View-Controller) et le modèle 3-tiers. L'application permet la gestion complète des produits avec un système d'authentification et de gestion des rôles.

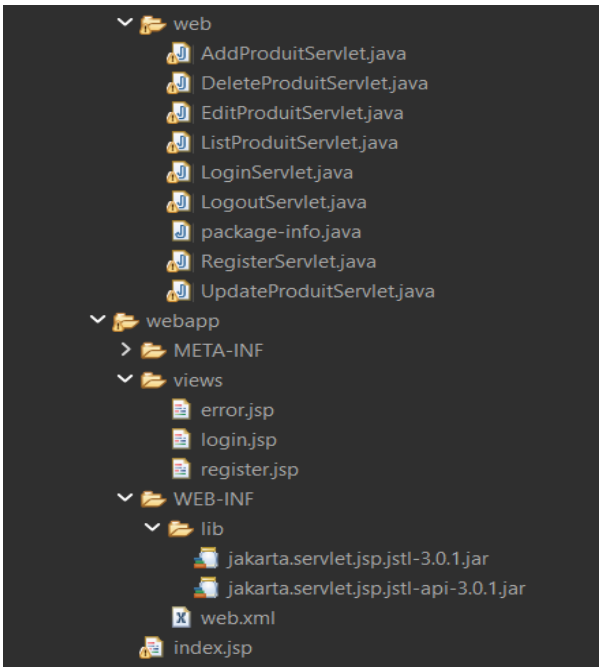
Objectifs du TP:

- Mettre en œuvre l'architecture MVC1 : une Servlet par requête
- Implémenter un système d'authentification avec sessions HTTP
- Gérer les rôles utilisateur (ADMIN / USER) via des filtres
- Utiliser JSTL pour séparer la présentation de la logique
- Concevoir une architecture extensible grâce aux interfaces DAO et Service

Architecture du projet:

Le projet suit une architecture 3-tiers combinée avec le patron MVC1 :





Couche Modèle (model/):

Les modèles sont des classes Java simples (POJO) sans aucune logique. Ils représentent les données échangées entre toutes les couches.

User.java:

```java
package model;

public class User {
    private Long id;
    private String username;
    private String password;
    private String email;
    private String phone;
    private String role; // "ADMIN" ou "USER"

    public User() {}

    public User(String username, String password, String email, String phone, String role) {
        this.username = username;
        this.password = password;
        this.email    = email;
        this.phone    = phone;
        this.role     = role;
    }
```

Produit.java:

```java
package model;

public class Produit {
    private Long id;
    private String nom;
    private String description;
    private Double prix;

    public Produit() {}

    public Produit(String nom, String description, Double prix) {
        this.nom           = nom;
        this.description   = description;
        this.prix          = prix;
    }
```

Couche DAO (dao/ et dao/impl/):

L'interface IProduitDAO joue un rôle fondamental dans l'architecture de l'application car elle respecte le principe Open/Closed (ouvert à l'extension, fermé à la modification). Ce principe stipule qu'une classe doit être **fermée aux modifications directes mais ouverte aux extensions**.

Concrètement dans notre projet :

La classe ProduitDAOImpl implémente l'interface IProduitDAO pour stocker les produits en mémoire (utilisation d'une Map). Si demain nous souhaitons passer à une véritable base de données (MySQL, PostgreSQL, etc.), il nous suffit de :

- Créer une nouvelle classe ProduitDAOMySQL qui implémente la même interface IProduitDAO
- Réécrire uniquement les méthodes (add, delete, update, findById, findAll) avec du code SQL
- Modifier une SEULE ligne dans le service (ProduitServiceImpl) pour utiliser la nouvelle classe.

IProduitDAO.java – Interface :

```java
package dao;

import model.Produit;

public interface IProduitDAO {
    void add(Produit produit);
    void delete(Long id);
    void update(Produit produit);
    Produit findById(Long id);
    List<Produit> findAll();
}
```

ProduitDAOImpl.java – Implémentation(Map en mémoire):

```java
package dao.impl;

import dao.IProduitDAO;

public class ProduitDAOImpl implements IProduitDAO {

    private static ProduitDAOImpl instance;
    private Map<Long, Produit> produits = new LinkedHashMap<>();
    private Long compteur = 1L;

    private ProduitDAOImpl() {
        add(new Produit("Clavier Mécanique",  "Cherry MX Red, rétro-éclairé RGB",     299.00));
        add(new Produit("Souris Gaming",       "Sans fil 16000 DPI, ergonomique",       149.00));
        add(new Produit("Écran 27\"",          "4K IPS 144Hz HDR400",                  1299.00));
        add(new Produit("Casque Audio",        "Bluetooth 5.0, réduction de bruit",     399.00));
    }

    public static synchronized ProduitDAOImpl getInstance() {
        if (instance == null) instance = new ProduitDAOImpl();
        return instance;
    }
    @Override
    public void add(Produit p) {
        p.setId(compteur++);
        produits.put(p.getId(), p);
    }

    @Override
    public void delete(Long id) { produits.remove(id); }

    @Override
    public void update(Produit p) { produits.put(p.getId(), p); }

    @Override
    public Produit findById(Long id) { return produits.get(id); }

    @Override
    public List<Produit> findAll() { return new ArrayList<>(produits.values()); }
}
```

Couche Service (service/ et service/impl/):

```java
package service;

import model.Produit;

public interface IProduitService {
    void addProduit(Produit produit);
    void deleteProduit(Long id);
    void updateProduit(Produit produit);
    Produit getProduitById(Long id);
    List<Produit> getAllProduits();
}
```

La couche Service contient la logique métier. Elle est appelée par les Servlets et appelle le DAO.

ProduitServiceImpl.java – Exemple de logique métier:

```java
package service.impl;

import dao.IProduitDAO;

public class ProduitServiceImpl implements IProduitService {

    private static ProduitServiceImpl instance;
    private IProduitDAO produitDAO = ProduitDAOImpl.getInstance();

    private ProduitServiceImpl() {}

    public static synchronized ProduitServiceImpl getInstance() {
        if (instance == null) instance = new ProduitServiceImpl();
        return instance;
    }

    @Override public void addProduit(Produit p)        { produitDAO.add(p); }
    @Override public void deleteProduit(Long id)       { produitDAO.delete(id); }
    @Override public void updateProduit(Produit p)     { produitDAO.update(p); }
    @Override public Produit getProduitById(Long id)   { return produitDAO.findById(id); }
    @Override public List<Produit> getAllProduits()     { return produitDAO.findAll(); }
}
```

Filtres HTTP (filter/):

Les filtres interceptent les requêtes AVANT qu'elles atteignent les Servlets. Ils permettent d'appliquer des contrôles de sécurité de manière centralisée.

AuthFilter.java – Vérification de la session:

```java
package filter;

import jakarta.servlet.*;

@WebFilter(urlPatterns = {
    "/listProduits", "/addProduit",
    "/editProduit",  "/updateProduit", "/deleteProduit"
})
public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest req, ServletResponse resp, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest  request  = (HttpServletRequest)  req;
        HttpServletResponse response = (HttpServletResponse) resp;

        HttpSession session = request.getSession(false);
        boolean connecte    = session != null && session.getAttribute("user") != null;

        if (connecte) {
            chain.doFilter(req, resp);
        } else {
            response.sendRedirect(request.getContextPath() + "/Login");
        }
    }
```

AdminFilter.java – Vérification du role:

```java
package filter;

import jakarta.servlet.*;

@WebFilter(urlPatterns = {"/addProduit", "/updateProduit", "/deleteProduit"})
public class AdminFilter implements Filter {

    @Override
    public void doFilter(ServletRequest req, ServletResponse resp, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest  request  = (HttpServletRequest)  req;
        HttpServletResponse response = (HttpServletResponse) resp;

        HttpSession session = request.getSession(false);
        String role = (session != null) ? (String) session.getAttribute("role") : null;

        if ("ADMIN".equals(role)) {
            chain.doFilter(req, resp);
        } else {
            response.sendRedirect(
                request.getContextPath() + "/listProduits?erreur=acces_refuse"
            );
        }
    }
}
```

L'ordre est garanti par la configuration web.xml.

Flux : Requête --> AuthFilter --> AdminFilter --> Servlet (chaque étape peut bloquer).

Configuration web.xml:

L'ordre des filtres est essentiel. Dans notre application, AuthFilter doit s'exécuter avant AdminFilter pour vérifier d'abord la connexion, puis les droits administrateur. Avec les annotations @WebFilter, l'ordre n'est pas garanti. Pour contrôler l'ordre, il faut déclarer les filtres dans web.xml dans l'ordre souhaité ou fusionner les deux filtres en un seul.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
             https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
         version="5.0">

    <display-name>GestionProduitsMVC1</display-name>

    <filter>
        <filter-name>AuthFilter</filter-name>
        <filter-class>filter.AuthFilter</filter-class>
    </filter>


    <filter>
        <filter-name>AdminFilter</filter-name>
        <filter-class>filter.AdminFilter</filter-class>
    </filter>


    <!-- AuthFilter s'applique à toutes les pages protégées -->
    <filter-mapping>
        <filter-name>AuthFilter</filter-name>
        <url-pattern>/listProduits</url-pattern>
        <url-pattern>/addProduit</url-pattern>
        <url-pattern>/editProduit</url-pattern>
        <url-pattern>/updateProduit</url-pattern>
        <url-pattern>/deleteProduit</url-pattern>
    </filter-mapping>
```

```xml
    <!-- AdminFilter s'applique seulement aux actions ADMIN -->
    <filter-mapping>
        <filter-name>AdminFilter</filter-name>
        <url-pattern>/addProduit</url-pattern>
        <url-pattern>/updateProduit</url-pattern>
        <url-pattern>/deleteProduit</url-pattern>
    </filter-mapping>


    <welcome-file-list>
        <welcome-file>listProduits</welcome-file>
    </welcome-file-list>

    <!-- ===== CONFIGURATION SESSION ===== -->
    <session-config>
        <session-timeout>30</session-timeout>
    </session-config>

    <!-- ===== PAGES D'ERREUR ===== -->
    <error-page>
        <error-code>404</error-code>
        <location>/views/error.jsp</location>
    </error-page>
    <error-page>
        <error-code>500</error-code>
        <location>/views/error.jsp</location>
    </error-page>

        https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd (xsi:schemaLocation with catalog)
</web-app>
```

Flux d'une requête HTTP (MVC1):

```
Navigateur
     |
     |-- GET/POST /addProduit
     v
AuthFilter  -->  Non connecté ?  -->  redirect /Login
     |
     |-- Connecté
     v
AdminFilter  -->  Pas ADMIN ?  -->  redirect
/listProduits?erreur=acces_refuse
     |
     |-- ADMIN
     v
AddProduitServlet  -->  ProduitServiceImpl  -->  ProduitDAOImpl
     |
     |-- sendRedirect /listProduits
     v
ListProduitServlet  -->  forward index.jsp  -->  HTML rendu
```

Couche Web – Servlets (web/):

Pour gérer les produits, l'application utilise plusieurs servlets spécialisées : AddProduitServlet (ajout), EditProduitServlet (préparation modification), UpdateProduitServlet (enregistrement modification), DeleteProduitServlet (suppression) et ListProduitServlet (affichage). Chaque servlet a une responsabilité unique : par exemple, **AddProduitServlet** récupère les données du formulaire HTML (nom, description, prix), crée un objet Produit et appelle le service ProduitServiceImpl pour l'ajouter. Cette séparation rend le code clair, maintenable et facile à faire évoluer.

```java
package web;

import jakarta.servlet.*;

@WebServlet("/addProduit")
public class AddProduitServlet extends HttpServlet {

    private IProduitService produitService = ProduitServiceImpl.getInstance();

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        try {
            String nom         = req.getParameter("nom");
            String description = req.getParameter("description");
            double prix        = Double.parseDouble(req.getParameter("prix"));
            produitService.addProduit(new Produit(nom, description, prix));
            resp.sendRedirect(req.getContextPath() + "/listProduits");
        } catch (NumberFormatException e) {
            req.setAttribute("error", "Le prix doit être un nombre valide.");
            req.setAttribute("listeProduits", produitService.getAllProduits());
            req.getRequestDispatcher("/index.jsp").forward(req, resp);
        }
    }
}
```

Servlet	URL	Méthode	Rôle
LoginServlet	/Login	GET	Affiche le formulaire de connexion
LoginServlet	/Login	POST	Authentifie → session + redirect
RegisterServlet	/Register	GET	Affiche le formulaire d'inscription

RegisterServlet	/Register	POST	Crée le compte → redirect ou erreur
LogoutServlet	/Logout	GET	Invalide la session + redirect /Login
ListProduitServlet	/listProduits	GET	Charge la liste (ou recherche) → forward index.jsp
AddProduitServlet	/addProduit	POST	Ajoute produit → sendRedirect /listProduits
EditProduitServlet	/editProduit	GET	Charge produit dans req. → forward index.jsp
UpdateProduitServlet	/updateProduit	POST	Met à jour → sendRedirect /listProduits
DeleteProduitServlet	/deleteProduit	GET	Supprime → sendRedirect /listProduits

Vues JSP (webapp/views/ et index.jsp):

Les JSP utilisent exclusivement JSTL (Jakarta Standard Tag Library) pour la logique d'affichage. Aucun scriptlet Java (<% %>) n'est utilisé, conformément aux bonnes pratiques MVC.

index.jsp – Extrait clé : formulaire dual Ajout / Modification:

```
<%-- ===== FORMULAIRE AJOUTER / MODIFIER (ADMIN uniquement) ===== --%>
<c:if test="${sessionScope.role == 'ADMIN'}">
    <div class="card mb-4 shadow-sm border-0">
        <div class="card-header bg-dark text-white">
            <c:choose>
                <c:when test="${not empty produitEdit}">
                    Modifier le produit #${produitEdit.id}
                </c:when>
                <c:otherwise>
                    Ajouter un produit
                </c:otherwise>
            </c:choose>
        </div>
        <div class="card-body">
            <%-- L'action du formulaire change selon le mode (ajout ou modification) --%>
            <form action="${pageContext.request.contextPath}/${not empty produitEdit ? 'updateProduit' : 'addProduit'}"
                  method="post">

                <%-- Champ caché : id du produit à modifier (vide si ajout) --%>
                <input type="hidden" name="idProduit" value="${produitEdit.id}">

                <div class="row g-2 align-items-end">
                    <div class="col-md-3">
                        <label class="form-label">Nom</label>
                        <input type="text" name="nom" class="form-control"
                               placeholder="Ex : Clavier"
                               value="${produitEdit.nom}" required>
                    </div>
                    <div class="col-md-4">
                        <label class="form-label">Description</label>
                        <input type="text" name="description" class="form-control"
                               placeholder="Ex : Mécanique RGB"
```

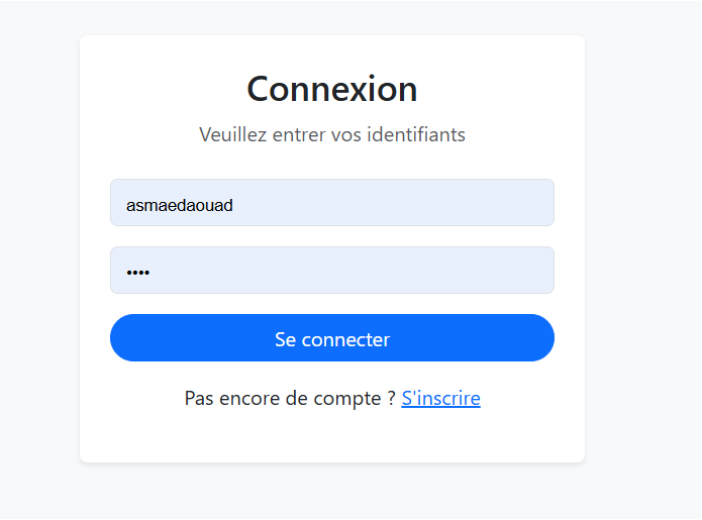
```jsp
                <input type="text" name="description" class="form-control"
                       placeholder="Ex : Mécanique RGB"
                       value="${produitEdit.description}" required>
            </div>
            <div class="col-md-2">
                <label class="form-label">Prix (DH)</label>
                <input type="number" step="0.01" min="0" name="prix"
                       class="form-control" placeholder="Ex : 299.00"
                       value="${produitEdit.prix}" required>
            </div>
            <div class="col-md-3 d-flex gap-2">
                <button type="submit" class="btn btn-primary flex-grow-1">
                    <c:choose>
                        <c:when test="${not empty produitEdit}">Confirmer</c:when>
                        <c:otherwise>Ajouter</c:otherwise>
                    </c:choose>
                </button>
                <%-- Bouton Annuler (visible uniquement en mode modification) --%>
                <c:if test="${not empty produitEdit}">
                    <a href="${pageContext.request.contextPath}/listProduits"
                       class="btn btn-outline-secondary">Annuler</a>
                </c:if>
            </div>
        </div>
    </form>
</div>
</div>
</c:if>
```

1. L'utilisateur"Admin" clique sur **"Modifier"** → lien href='/editProduit?id=3'
2. **EditProduitServlet** récupère le produit correspondant et fait req.setAttribute("produitEdit", p)
3. La requête est **forward** vers index.jsp → le JSP détecte que produitEdit != null
4. Le formulaire est alors **pré-rempli** avec les valeurs du produit à modifier
5. L'action du formulaire devient updateProduit au lieu de addProduit

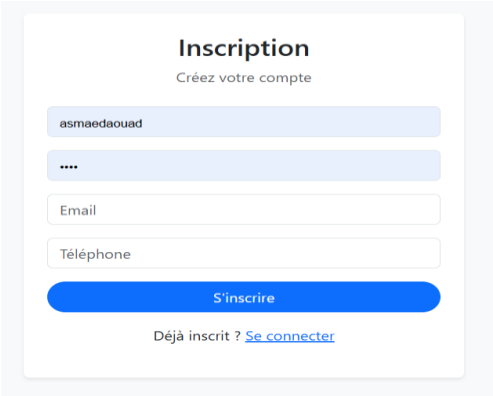
Interfaces de l'application:

Les captures d'écran ci-dessous illustrent les principales vues de l'application. Elles sont à insérer manuellement depuis votre navigateur.

Page de connexion (login.jsp):



Page d'inscription (register.jsp):



Page principale – Vue ADMIN (index.jsp):

Page principale – Vue USER (index.jsp) :

Gestion des Produits

Bonjour, **asmaedaouad** [USER] Déconnexion

Rechercher par ID | Rechercher | Réinitialiser

ID	Nom	Description	Prix
1	**Clavier Mécanique**	Cherry MX Red, rétro-éclairé RGB	299.0 DH
2	**Souris Gaming**	Sans fil 16000 DPI, ergonomique	149.0 DH
3	**Écran 27"**	4K IPS 144Hz HDR400	1299.0 DH
4	**Casque Audio**	Bluetooth 5.0, réduction de bruit	399.0 DH

Conclusion:

Ce TP nous a permis de mettre en pratique les concepts fondamentaux du développement web Java EE avec l'architecture MVC1 et le modèle 3-tiers.

Compétences acquises:

- Maîtrise de l'architecture MVC1 : une Servlet par action, séparation claire des responsabilités
- Utilisation des sessions HTTP pour gérer l'état utilisateur dans un protocole stateless
- Implémentation des filtres HTTP pour une sécurité transversale (AuthFilter, AdminFilter)
- Utilisation de JSTL pour une séparation stricte entre la logique et la présentation
- Conception avec interfaces DAO et Service pour une architecture extensible
- Compréhension du pattern Singleton pour les données partagées en mémoire

Perspectives d'amélioration:

- Migration vers JDBC avec MySQL pour la persistance des données (V2)
- Ajout d'un hashage des mots de passe (BCrypt) pour la sécurité
- Pagination du tableau des produits avec des paramètres GET
- Migration vers MVC2 (Spring MVC) pour comparer les deux architectures